

BuildingElement

Description

The BuildingElements are the actual objects placed by the Advanced Building System. The Buildings are created and built up from the BuildingElements.

A BuildingElement can be a building piece of a structure like a wall or a floor element. It can be a door or a window. It can be furniture like a table or a chest. It can be a tree or a turret. It just depends on you and your project.

Setup

You need to make your own BuildingElements for your game. You can do it by yourself or you can use the Advanced Building System's custom tool BuildingElementCreator which will be discussed in the Tools documentation.

Every BuildingElement should have the following components:

- Box Collider
- BuildingElement (Script)

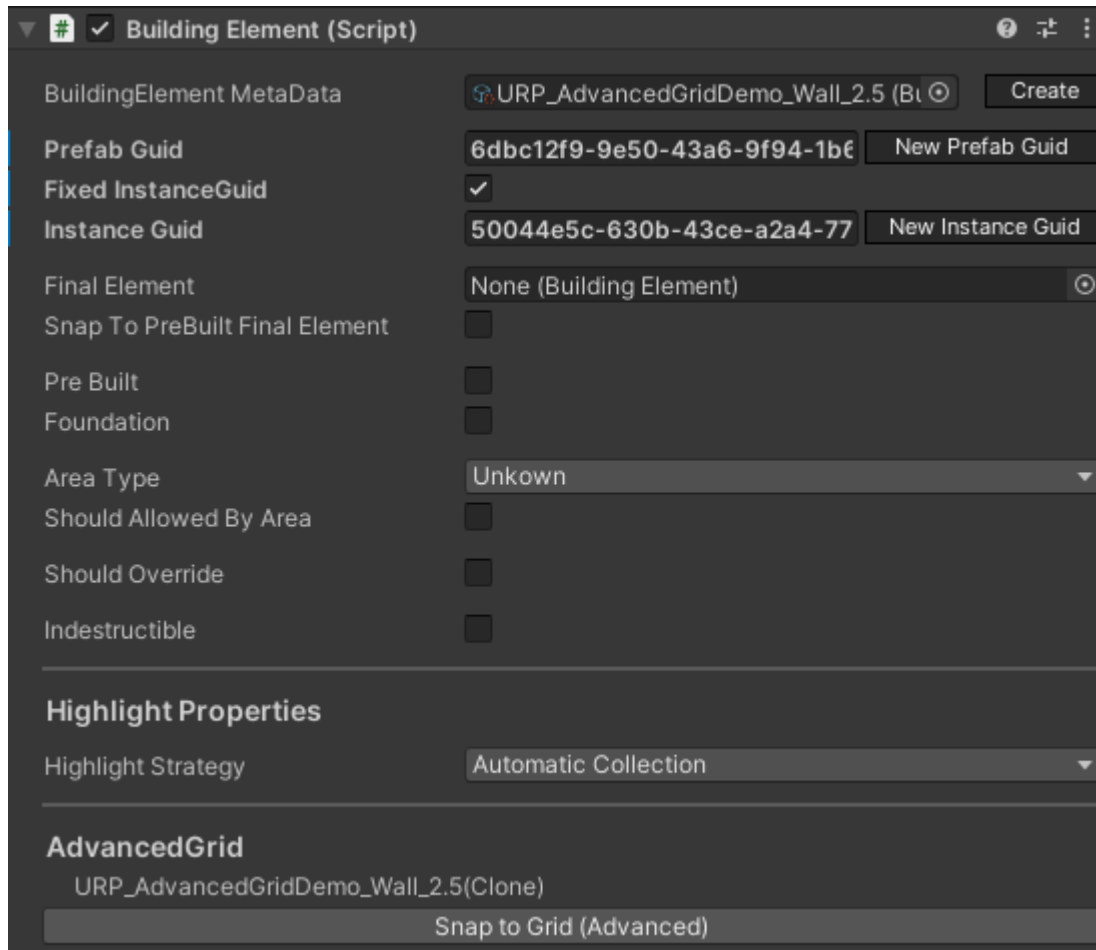
Box Collider

The Building Element should have a Box Collider component. It is used by the algorithm. The size of the collider is recommended to match with the size of the Building Element that is set in the MetaData as Dimension.

The "Include Layers" should contain the BuildingElement and the Player's layer.

Properties Of BuildingElement

The BuildingElement script has some property that is specific for that actual element. Most of the properties of a BuildingElement is stored in a BuildingElementMetaData for easier reusability.



MetaData is a collection of properties for BuildingElements. Multiple MetaData can be created and one MetaData can be used for multiple BuildingElements for making it easier to set up similar elements that should have the same properties and the BuildingManager should handle these elements in the same way. Like 2 different walls should be used in a same way so the MetaData can be shared but some specific property can be set directly on the BuildingElement.

The **Prefab Guid** parameter is used for identifying a BuildingElement. The Guid is important to be unique for every different element.

Note that the Prefab Guid is for the prefab of the BuildingElement. Not the actual instance of the element. It is unique for the prefab but not for the instance.

The “New Prefab Guid” button can be used for generating new unique guid.

For example if you copy a BuildingElement the properties of the script will be the same just like the Prefab Guid. So the algorithm will think that these are the same BuildingElement based on that guid. To fix this issue if you copied the BuildingElement regenerate the guid.

Instance Guid: The InstanceGuid is meant to be unique for every single Instance of any BuildingElement. This is used to identify the actual instance of the BuildingElement, not the element's prefab. It is used for the History feature and makes the handling of the BuildingElement easier from the tracker if every instance can be identified with a Guid. The Guid is generated in the construction of the BuildingElement or it is created and overridden by the algorithm if the element is recreated by the persistence or the history features to ensure that the destroyed (undo) and later replaced (redu) element has the same element as the original one.

Fixed InstanceGuid is a boolean and if it is true then the InstanceGuid can be overridden by hand from the Unity's inspector view for ensure that the guid is predictable what can be used for example for the elements what already placed by the developers and it make it easier to identify them during runtime.

The **Final Element** is a BuildingElement that will be built instead of the actual BuildingElement. It is working like that during the build process the algorithm will use the actual element but when the element is finalized so the player releases the button to build the algorithm and replace the current element to the final element.

In detail the algorithm checks at the end of the build process if the BuildingElement has a Final Element (it is not null) then destroys the current element and instantiates one from the Final Element and places it right in the actual element's position. It is working in simple building and drag building scenarios too. The main idea behind this feature is that you can use another element during the build process. Like you can use a half constructed wall but when you build it the whole wall will be placed. Or you can use a generic element like a cube during the build process and place a table instead of that.

Snap To Pre Built Final Element: When you are building the current element the algorithm will try to snap your element into a Pre Built element if it is nearby. It is only working when the element that was used to build is the same as the Pre Built element. So if you are using a Final Element and you are placing a half constructed wall but the final element is a whole wall the algorithm will snap your element only into the half constructed pre built wall and it can not snap into the whole pre built wall even if it will build a whole wall at the end of the process instead of the half wall. If the "Snap To Pre Built Final Element" boolean is true the algorithm allows you to snap your element into a pre-built element from your final element.

The **Pre Build** property is an important parameter that can not be set by default so if you are adding a BuildingManager to the scene you should set this property by hand. It is a boolean which means that the BuildingElement should be handled as a PreBuilt element or not. The PreBuilt Element is explained in detail later in the documentation.

Foundation is also a boolean. The BuildingManager's has the "Use Foundation Logic" property. If the BuildingManager's property is true then a BuildingElement can be built only if it is a foundation or if it is snapping to another already built element.

AreaType is used for Area features that will be explained in detail later. With this enum you can set the type of the BuildingElement for the Area feature.

Should Allowed By Area means that if it is true the BuildingManager will block the building of that element until any of the BuildingArea doesn't allow it.

The **Should Override** means that if it is true the BuildingManager will block the building of that element until it is overriding another element with the Override Element feature.

Note that this feature doesn't allow the element to snap to an PreBuilt element because that scenario is not overriding.

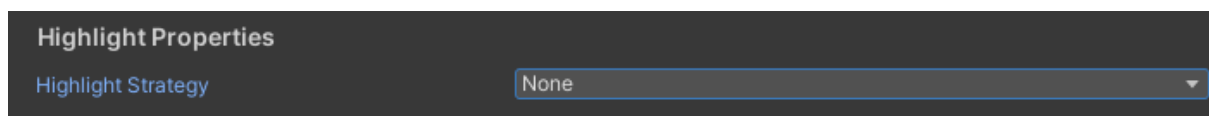
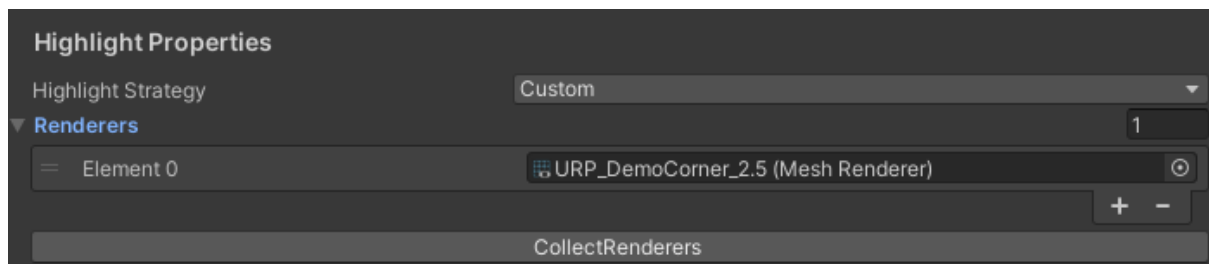
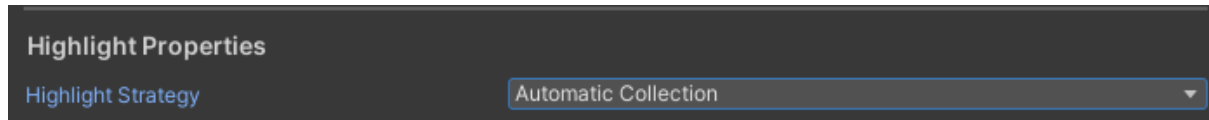
Indestructible is a boolean value. If it is true then the destory functionality will ignore the BuildingElement so it can not be destroyed by the BuildingManager. Also it will not be highlighted.

Note that this ignore means that the element is handled like the raycast hit something that isn't a BuildingElement. If you are using the Destroy feature with Timer and the "Cut Timer On Look Away" enabled the timer will be stopped when the player looks to an Indestructible element because it is handled like it is not a BuildingElement.

Highlight Properties

The BuildingManager uses a highlight feature that will change or modify the materials of the renderers of the BuildingElement. It is used to highlight the different states of the BuildingElements like blocked or pre built etc.

You can select from the following 3 behavior:



None: There will be no highlighting. The Building Element will get the event and change its states but doesn't change the materials at all. If you don't want the highlight feature just set this property to None.

Automatic Collection: The BuildingElement will collect the rendered from its game object and its childs and use the highlight logic so it will modify the materials of the renderer. Note that this will collect every renderer and change these materials.

Custom: It is basically the same as the Automatic Collection. The Building Element will use the highlight feature and it will change the material of the rendere just in this time you can set by hand which renderers should be used. To make it easier to collect the renderers of the GameObject the CollectRenderers button can be used for automatically collecting the renderers just like the Automatic Collection. Just in this time you can modify the list of the elements by hand.

BuildingAlgorithm Specific properties

Snap to Grid

“Snap to Grid (Advanced)” and “Snap to Grid (Basic)” Buttons are the only available algorithm specific options of the BuildingElements. These 2 buttons can be used during the scene building process when the developer is placing the BuildingElements by hand and he/she wants to ensure that the elements are snapping to the grid. Just place the BuildingElement to the scene and press the button. It will modify its transform to snap into the grid. Note that this feature is only working if the BuildingElement's parent object has the proper type of Building script as component.

In case of multi-object editing (multiple elements are selected) the user interface will show a list about the elements that will be affected by the “Snap to Grid” feature.

The screenshot displays the 'Building Element (Script)' configuration window. It features a dark-themed interface with several sections:

- BuildingElement Metadata:** Includes a dropdown menu and a 'Create' button.
- Prefab Guid:** Displays 'Multiply target, can't show the Guid!' with a 'New Prefab Guid' button.
- Fixed InstanceGuid:** Displays 'Multiply target, can't show the Guid!'.
- Final Element:** A dropdown menu currently set to 'None (Building Element)'.
- Snap To PreBuilt Final Element:** A checkbox that is currently unchecked.
- Pre Built:** A dropdown menu currently set to '-'. Below it is a 'Foundation' checkbox, which is also unchecked.
- Area Type:** A dropdown menu currently set to 'Unkown'.
- Should Allowed By Area:** An unchecked checkbox.
- Should Override:** An unchecked checkbox.
- Indestructible:** An unchecked checkbox.

Highlight Properties

- Highlight Strategy:** A dropdown menu currently set to 'Automatic Collection'.

AdvancedGrid

- URP_AdvancedGridDemo_Wall_2.5(Clone)
- URP_AdvancedGridDemo_Wall_2.5(Clone)
- URP_AdvancedGridDemo_Wall_2.5 1 (1)

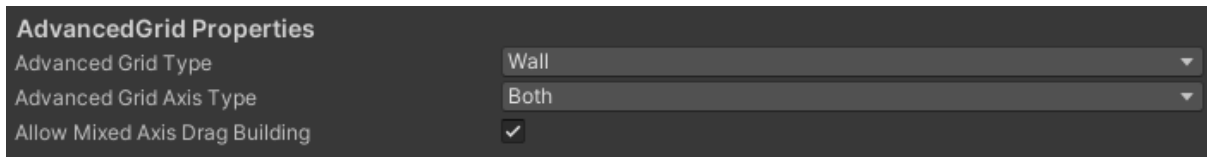
Snap to Grid (Advanced)

BasicGrid

- URP_BasicGridDemoFloor_2.5 (3)
- URP_BasicGridDemo_Cube_2.5

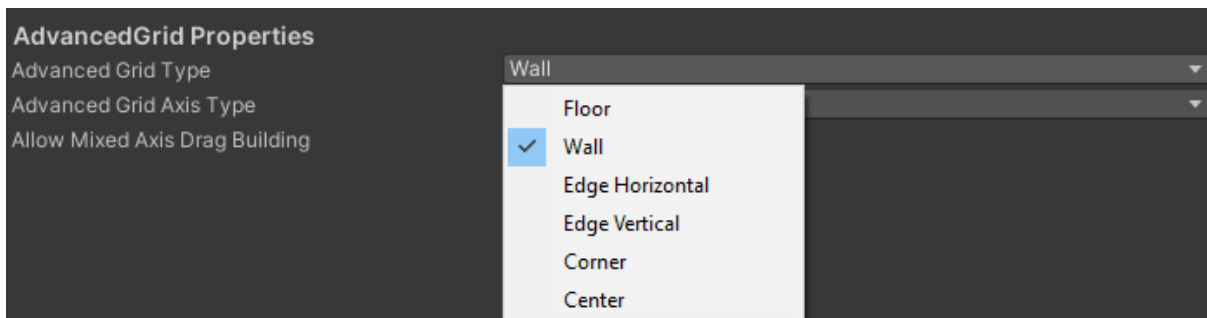
Snap to Grid (Basic)

Advanced Grid Building

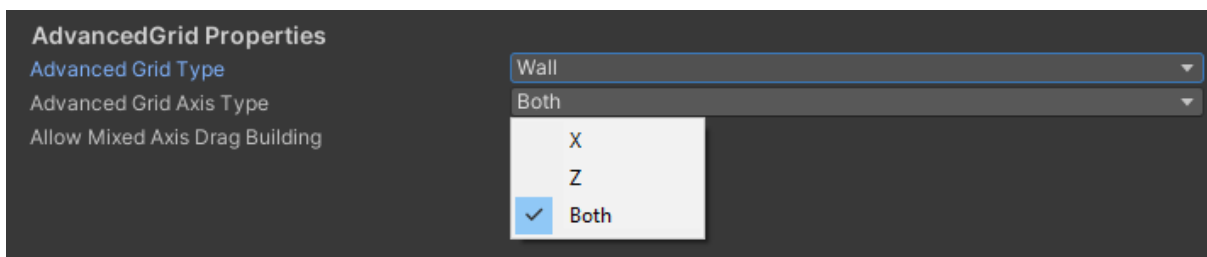


The BuildingElement MetaData has some specific properties for the AdvancedGrid Building algorithm.

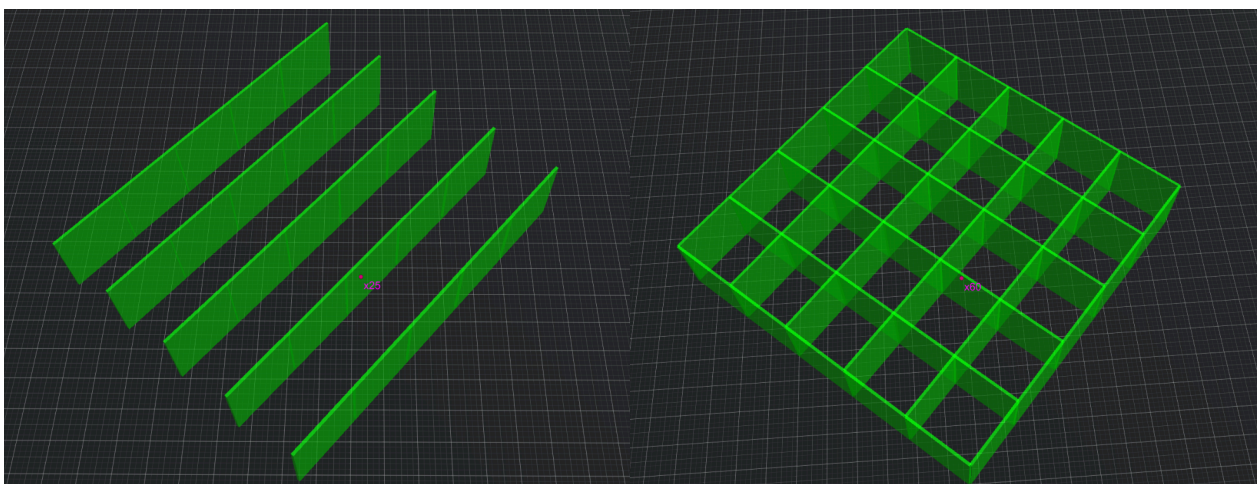
First the Advanced Grid Type so what kind of element is for this MetaData.



In case of Wall or Horizontal Edge the Advanced Grid Axis Type should be set too. The developers can force the building of this element to only one axis.



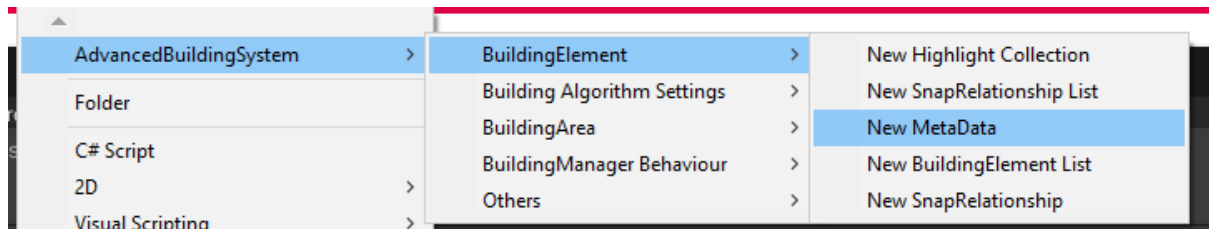
The Allow Mixed Drag Building is also available for the Wall and the Horizontal Edge. If this is true then The algorithm will use both axes during the Drag Building.



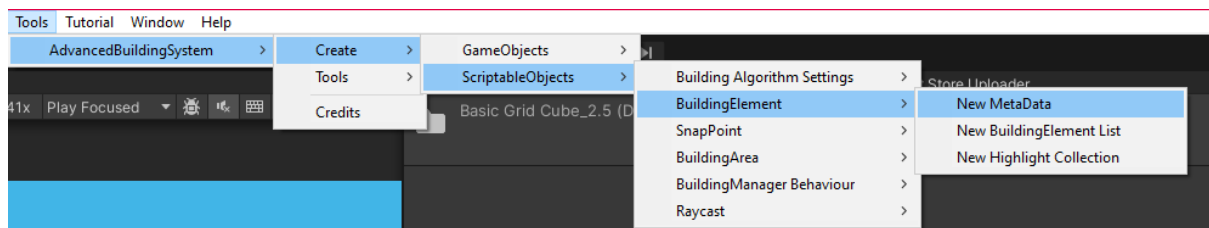
BuildingElement MetaData

For easier setup of the similar BuildingElements that should be handled by The Manager in the same way most of the properties are separated from the element objects into ScriptableObjects.

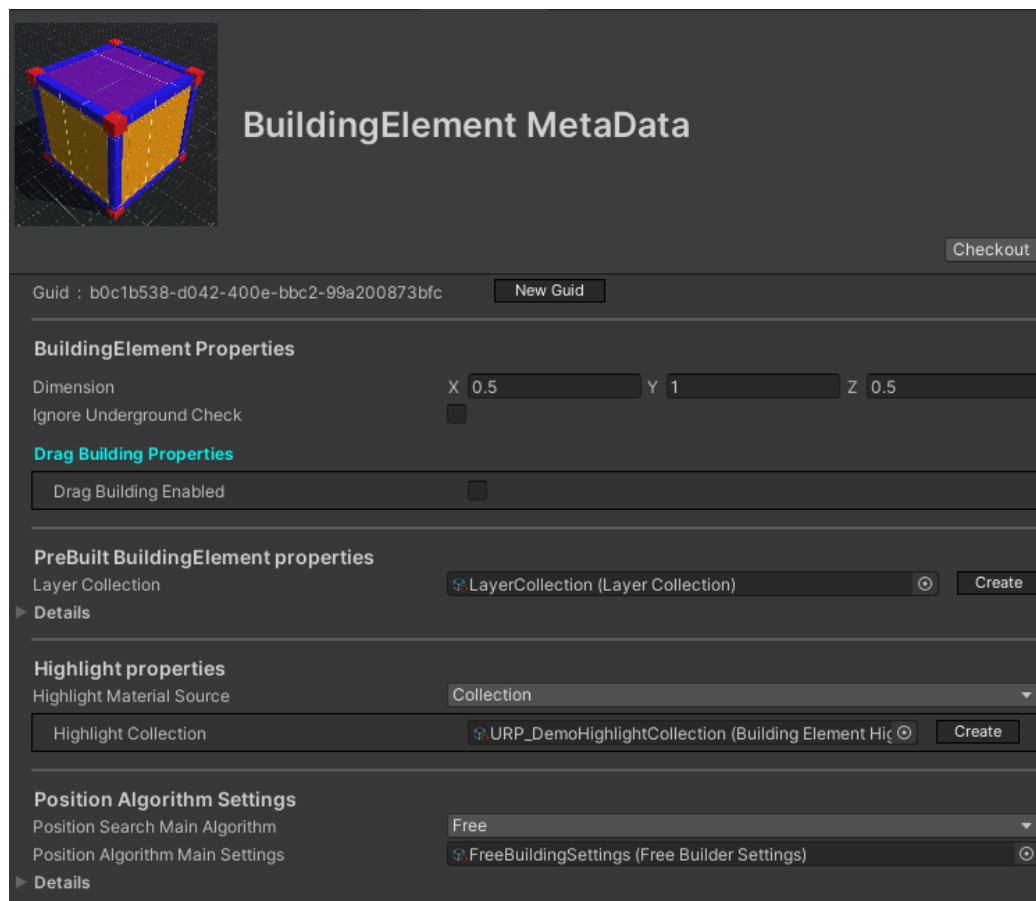
To create a BuildingElementMetaData right click on the Project view:



Or you can create it from the menu bar:

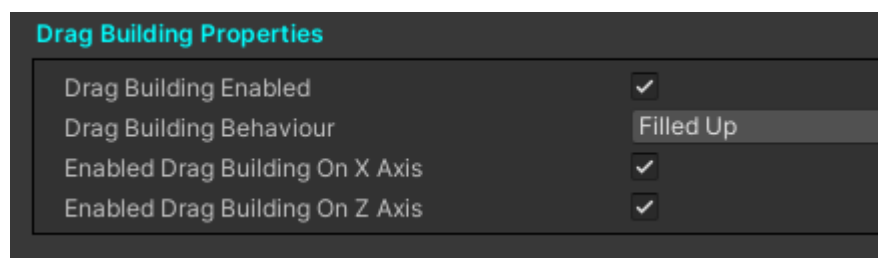


Building Element MetaData GUI:



The MetaData has the following properties:

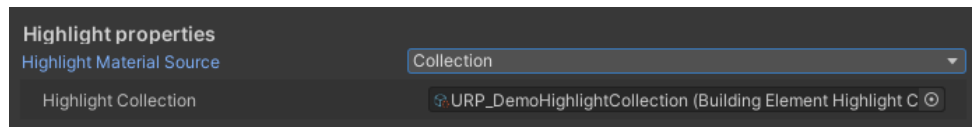
- BuildingElement Properties
 - Guid
 - Can not be set. (It is set automatically when the MetaData has been created)
 - It should be unique.
 - It is used for identifying the BuildingElements with the same MetaData. It is different from the Guid of the BuildingElement's script. That Guid is for identifying the concrete element.
 - **Note that if you copy the ScriptableObject its properties are copied too. Ensure that the Guid has been regenerated with the "New Guid" button.**
 - Dimension
 - It is used as the size of the BuildingElement. Ensure that it has been set properly because all the calculations are based on this Vector3.
 - For example if you are using BoxCollider for the BuildingElements use the same size for the Dimension as the Vector3 used for the BoxCollider.
 - Ignore UnderGround Check
 - The BuildingManager is trying to avoid placing the elements under the ground where the element can not be seen. If you set this boolean the BuildingManager will allow you to place this element under the ground.
 - Drag Building Properties
 - DragBuilding is one of the main features of the BuildingManager. It is about placing multiple elements at the same time.
 - Here you can set up how the DragBuilding algorithm should handle this element during the drag building process or you can just disable the drag building feature on this element so the BuildingManager will not allow the DragBuilding with this element.



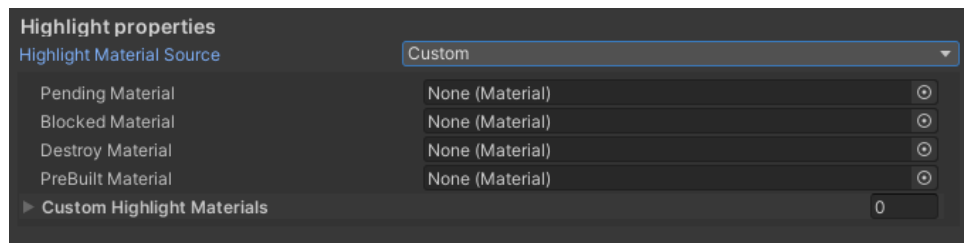
- If you enable the DragBuilding feature you can set its behaviour. What will be discussed later.
 - Next to that the drag building can be disabled separately on the axis.
- PreBuilt BuildingElement properties
 - Layer Collection
 - The Layer Collection holds the layer of the player for the PreBuiltElement feature.
- Highlight Properties
 - The Materials used for highlighting the BuildingElements in different scenarios during the BuildingProcess. It is not commonly set on the BuildingManager because you can set it for every MetaData to make it possible to set different

highlight material for different elements.

- The effect on the BuildingElements are partially explained during the BuildingElement properties where you can set which renderers should be affected by the highlight feature. Here you can set the materials for that feature.
- You have two options:
 - Collection: You should provide a HighlightCollection ScriptableObject which is just a collection of materials for different states for the BuildingElement.



- Custom: You should provide the materials right on the MetaData ScriptableObject.



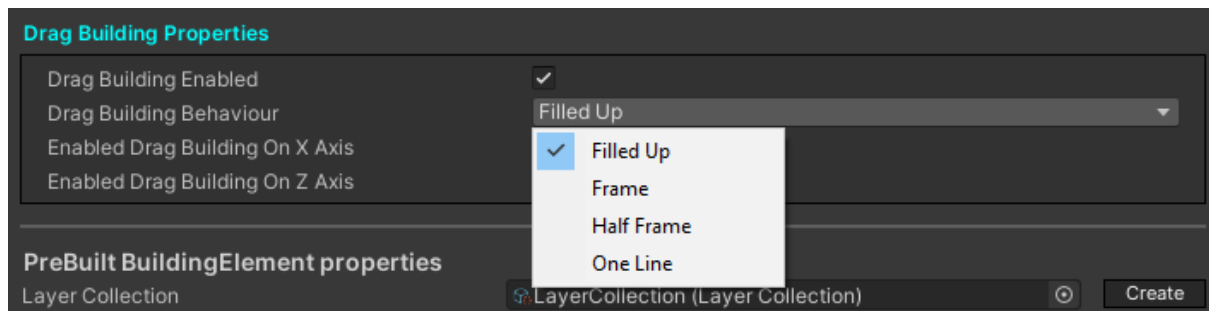
- What's the purpose of the Custom Highlight Materials?
 - You can set a list of materials that should be used as highlights on the BuildingElement in different scenarios that are independent from the AdvancedBuildingSystem but you want to achieve the same highlight mechanism as the built in scenarios.
 - For example you want to outline the BuildingElement if your cursor is above an element. You should add the material in the list and call the **SetHighlightMaterial** on the BuildingElement with the index of the Highlight material. In this case the index of the outline's material.
- Position Algorithm Settings
 - Every BuildingElement can be built in a different way using different algorithms. For every kind of BuildingElements used in your project you can set different building algorithms.
 - Some algorithms need a fallback algorithm that has been set.
 - Every algorithm needs a setting that can be created as the MetaData (ScriptableObject) Different settings can be created for every MetaData to make it possible to use different build logic for different elements.
- Position Algorithm Specific Settings.
 - Some of the algorithms have their own unique settings that can be set on the Building Element MetaData but they will be discussed later with the specific algorithm.

DragBuilding Behaviour

You have the following options:

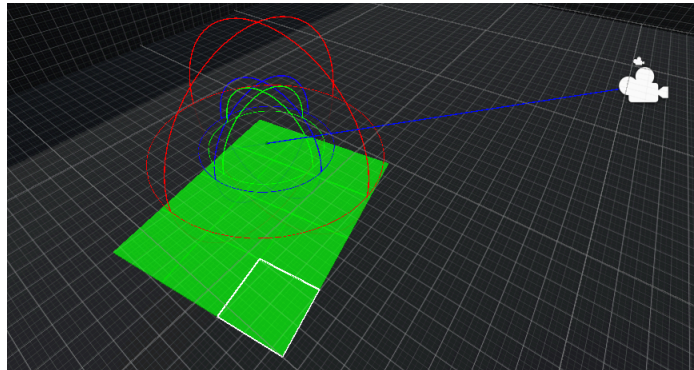
1. Filled Up
2. Frame
3. Half Frame
4. One Line

With these options you can change the shape of the drag building results.



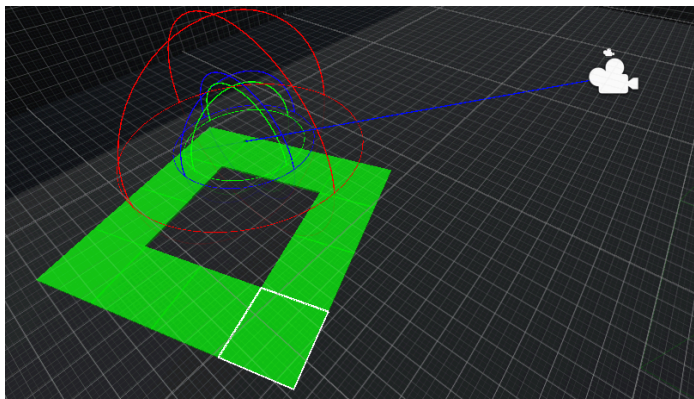
Filled Up:

With the Filled Up setup the algorithm will fill up the whole area. So every line and every row will be filled up with elements.



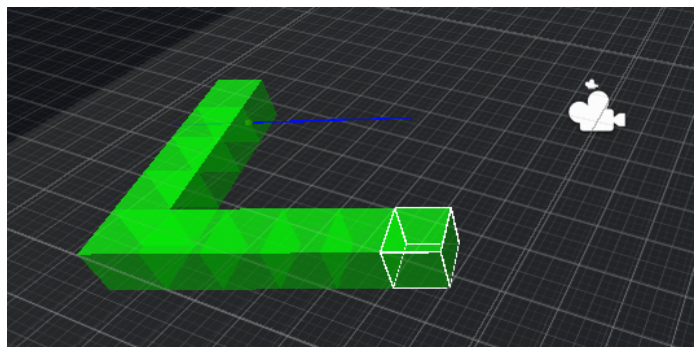
Frame:

With the Frame Setup the algorithm will place the elements only on the edge of the area. The inner elements are not placed.



Half Frame:

It is basically the same mechanism as the Frame, it just only places the elements on the longer side.

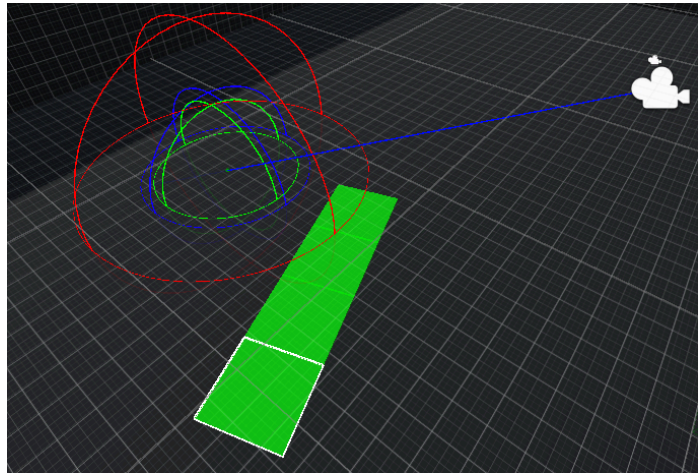


One Line:

What is the difference from the OneLine setup and the limit of an axis? If you limit one of the axes then the BuildingManager will build only on the other axis. But with this setup you can build a direct line on any axes.

The algorithm will check the raycast's results and it will build the line on the X or Z axes which one is nearest to the raycast's hit point.

So with OneLine you can get a direct line on any of the axes.

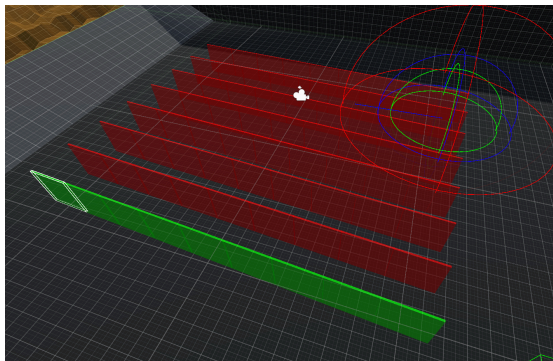


DragBuilding Behaviours + Mixed Axis

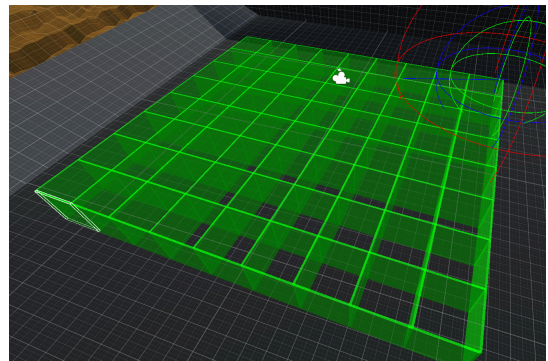
In this chapter you can see how the different DragBuilding Behaviours work with mixed axis in case of Advanced Grid Building and a wall.

Note: in this examples some cases the elements on the Z axis doesn't touch the building so they are blocked

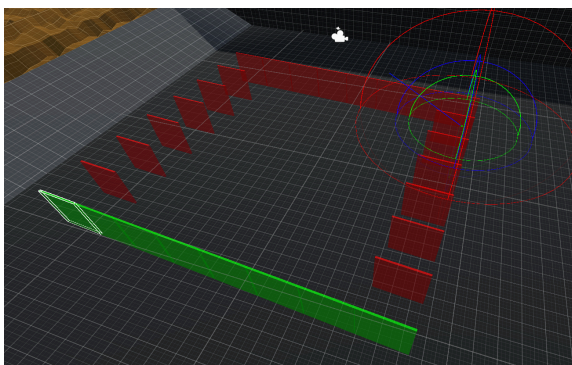
Filled Up



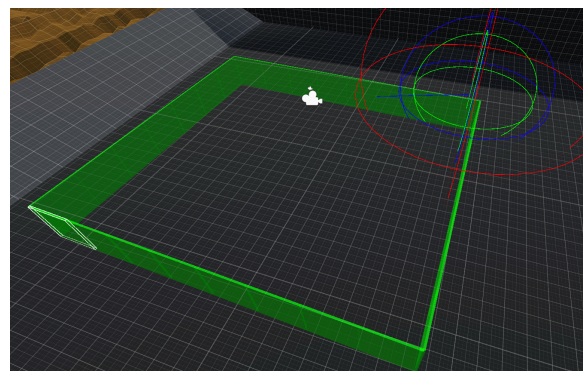
Filled Up + Mixed Axis



Frame

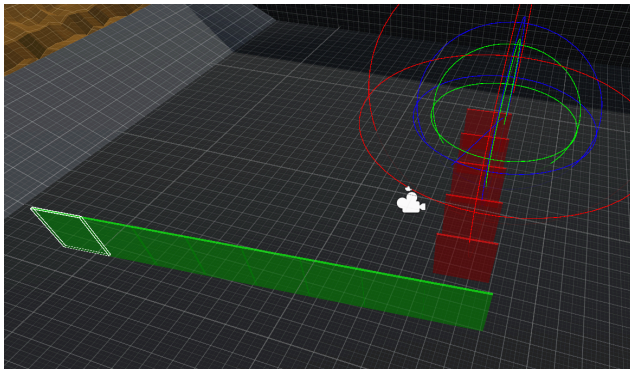


Frame + Mixed Axis

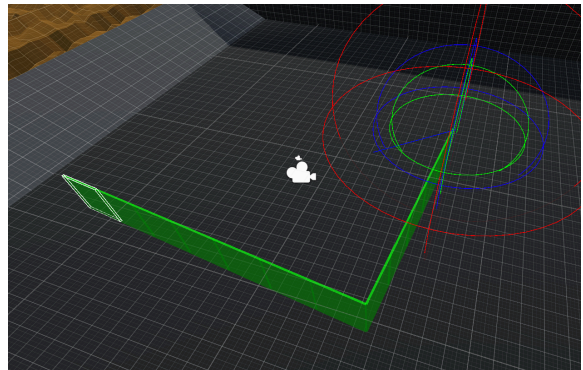


Half Frame

(X is the longer Side)

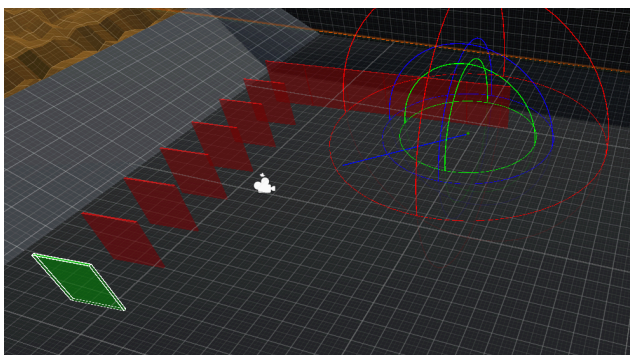


Half Frame + Mixed Axis

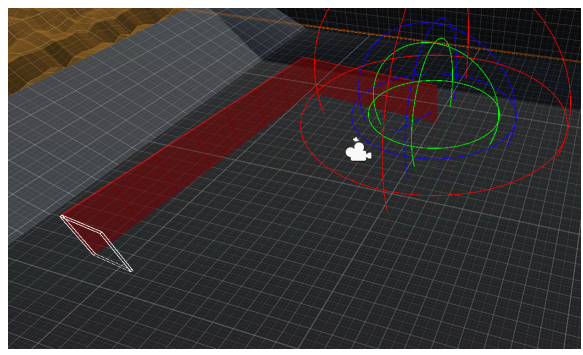


Half Frame

(Z is the longer Side)

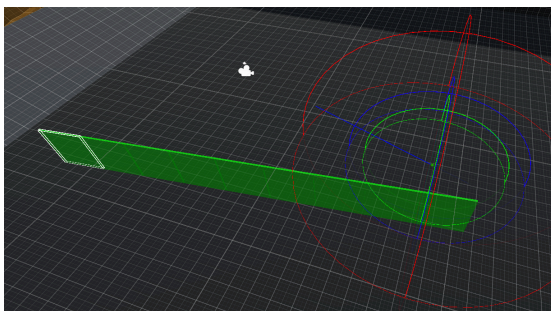


Half Frame + Mixed Axis

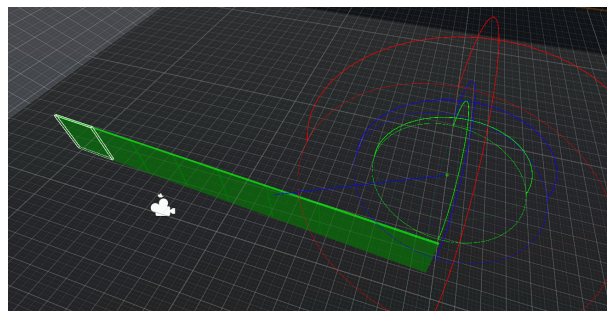


OneLine

(X is the longer Side)

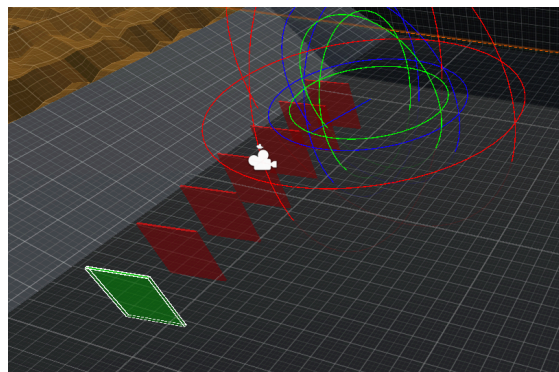


OneLine + Mixed Axis

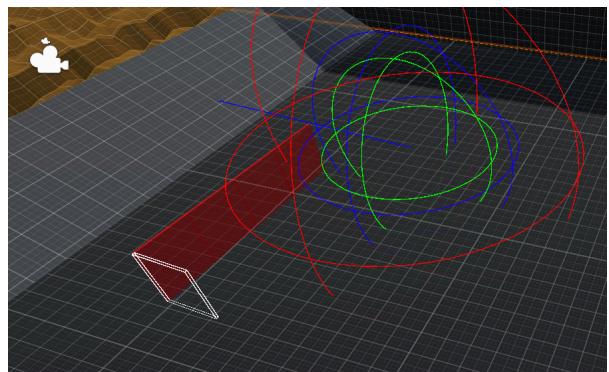


OneLine

(Z is the longer Side)



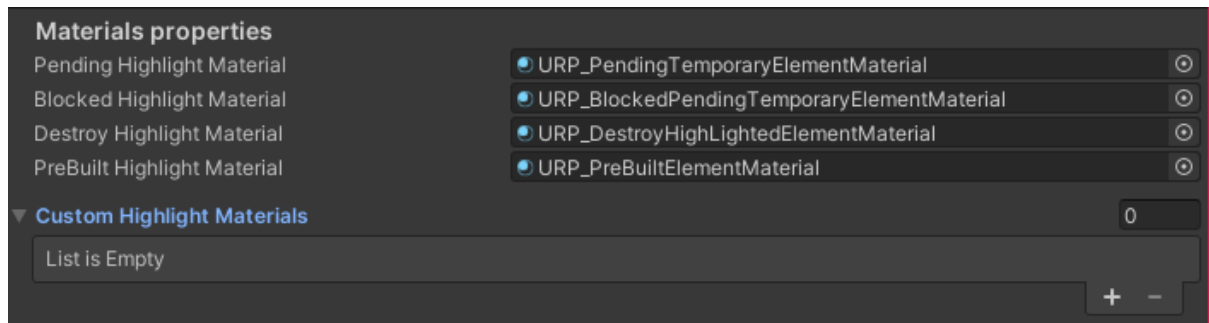
OneLine + Mixed Axis



Highlight Collection

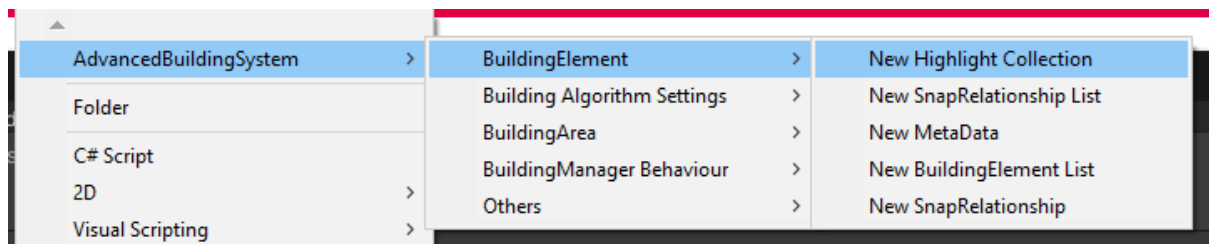
The HighlightCollection is just a list of the materials that should be used for highlighting the BuildingElements in different states. And also a list of different materials for custom highlighting the BuildingElements.

This custom highlight feature can be done by calling the **SetHighlightMaterial** function on the BuildingElement script and providing the index of the material from the “Custom Highlight Materials” list.

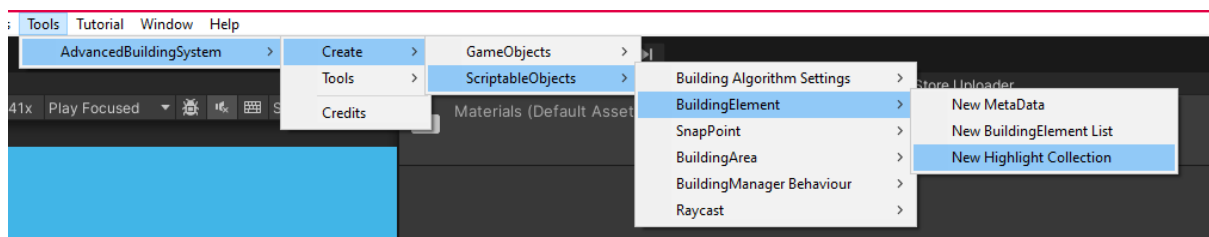


The Highlight Collection can be created just like every other scriptable object used by the BuildingManager.

From the project:



From the Menu:



IBuildingElementExternalInterface

The BuildingElement has a lot of functions that are used by the algorithm but some of them are important to the developers who are using this asset. To separate the functions that are meant to be used by the users of the asset the BuildingElement is implementing the IBuildingElementExternalInterface interface. This interface is a collection of public functions that are safe to use by the developers. Do not recommend using any other function on the BuildingElement because it can easily break the logic of the Advanced Building System.

You have the following functions:

- **Getters / Setters**
 - Metadata
 - FinalElement
 - Renderers
 - Collider
 - SnapToPreBuiltFinalElement
 - Foundation
 - ShouldAllowedByArea
 - ShouldOverride
 - Indestructible
 - AreaType
 - Dimension
 - ParentBuilding
 - CanSnapUnderground
 - DragBuildingEnabled
 - EnabledDragBuildingX
 - EnabledDragBuildingZ
 - AdvancedGridType
 - AdvancedGridAxisType
 - PreBuilt
- **Functions**
 - SetHighlightMaterial
 - SetMaterialToDefault
 - Destroy

PreBuilt BuildingElement

Sometimes that can be useful if some BuildingElement is already in the scene to show to the player where that element should be placed.

So what is a PreBuilt BuildingElement? The PreBuiltBuildingElements are basic BuildingElements but

- it can not collide with the player
- it has a special highlight material
- it handled specially by the AdvancedGridBuilding
 - The BuildingElements can not snap into its position just if it is the same element.
 - The BuildingElements can not snap to a position if every neighboring BuildingElement is PreBuilt.
 - The DragBuilding should handle the PreBuilt elements.
 - If a PreBuilt element from the same BuildingElement is nearby the algorithm will prioritize its position.
 - The BuildingElements can not be validated by neighbor PreBuiltBuildingElements.
- the PreBuiltBuildingElement should be destroyed if a right element is snapped into its position.

The main goal is that the developers are able to make such BuildingElements that can not be collided by the player because they are “not placed yet”. When the player is building something and a PreBuilt element with the same type is nearby the element should be snapped right into its position.

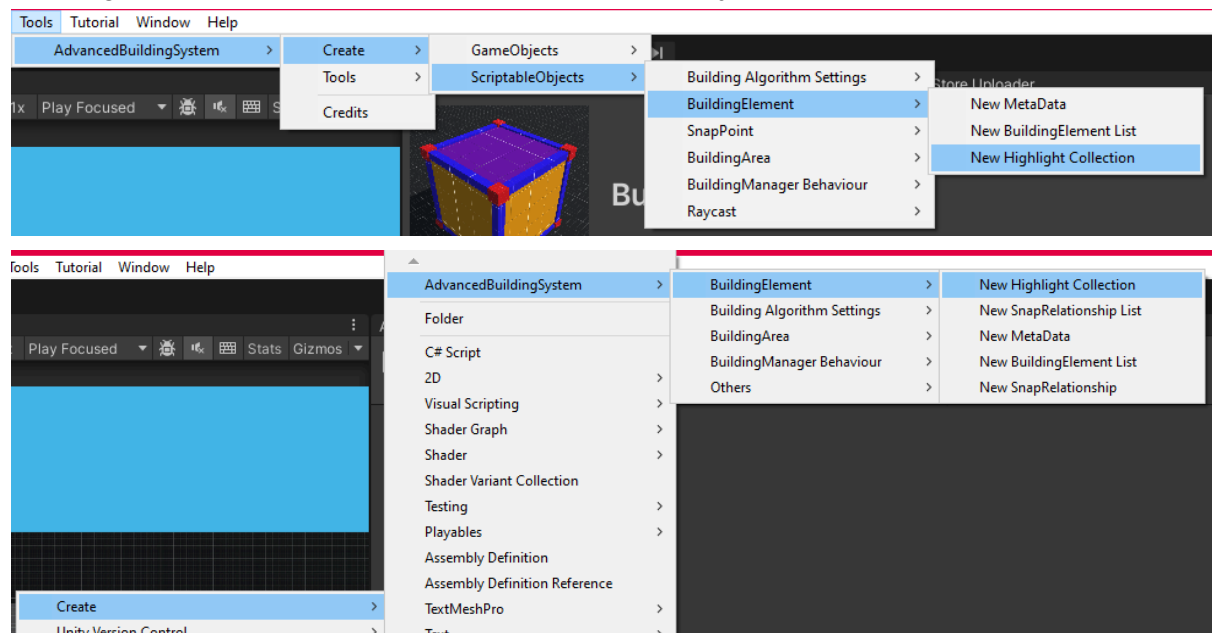
BuildingElementList

Currently the BuildingElementList is just a list of BuildingElements. It should be provided to the BuildingManager. The manager will use it to get the BuildingElement GameObjects to instantiate the elements.

When the BuildingManager is activated through the IBuildingManagerExternalInterface interface an index of the BuildingElement from the list should be provided to the Manager. The list can be changed during the game using the **SetBuildingElementList** function of the earlier mentioned interface.

This list is used by the History feature. If an element is not contained by the list the undo/redo features will fail.

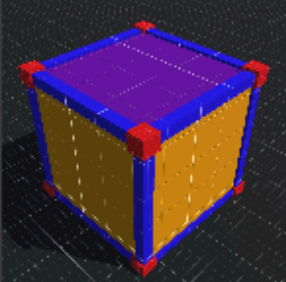
Creating the List can be done from the Menu or the project Create option:



The secondary feature of the BuildingElementList is a validation action that can be triggered on the UI. The validation will try to check some issues and give feedback for the developers. The following will be checked:

- Null element in the list. (None or Missing elements)
- Duplicated elements (Same PrefabGuid)
- Elements without MetaData (The MetaData is null)

In the following image you can see a failed validation that is trying to show the possible issues.



BuildingElementList

Checkout

▼ BuildingElements

9

Element 0	URP_AdvancedGridDemo_Wall_2.5 (Building Elem
Element 1	None (Building Element)
Element 2	None (Building Element)
Element 3	URP_AdvancedGridDemo_Wall_2.5 (Building Elem
Element 4	URP_AdvancedGridDemo_Wall_2.5 1 (Building Elei
Element 5	URP_AdvancedGridDemo_WallDoor_2.5 (Building
Element 6	Missing (Building Element)
Element 7	Missing (Building Element)
Element 8	URP_AdvancedGridDemo_Wall_2.5 2 (Building Ele

+ -

Validate

! Invalid Object at : 1, 2, 6, 7

! The following elements are sharing the same PrefabGuid!

PrefabGuid: e4ea9504-6c96-4160-ab00-fb27582643bb

0		URP_AdvancedGridDemo_Wall_2.5
3		URP_AdvancedGridDemo_Wall_2.5
4		URP_AdvancedGridDemo_Wall_2.5 1
5		URP_AdvancedGridDemo_WallDoor_2.5

! The following elements hasn't MetaData

8		URP_AdvancedGridDemo_Wall_2.5 2
---	---	---------------------------------